



SESSION 2

First Programs and the Shell

Strings and Expressions

Arewa Data Science Academy

Strings, Assignments, and Comments

- Text processing is by far the most common application of computing
- E-mail, text messaging, Web pages, and word processing all rely on and manipulate data consisting of strings of characters

Data Types (1 of 2)

- A **data type** consists of a set of values and a set of operations that can be performed on those values
- A **literal** is the way a value of a data type looks to a programmer
- `int` and `float` are **numeric data types**
 - They represent numbers

Data Types (2 of 2)

Type of Data	Python Type Name	Example Literals
Integers	<code>int</code>	-1, 0, 1, 2
Real numbers	<code>float</code>	-0.55, .3333, 3.14, 6.0
Character strings	<code>str</code>	"Hi", "", 'A', "66"

String Literals (1 of 2)

- In Python, a string literal is a sequence of characters enclosed in single or double quotation marks
- ' ' and " " represent the **empty string**
- Double-quoted strings are handy for composing strings that contain single quotation marks or apostrophes:

```
>>> "I'm using a single quote in this string!"  
"I'm using a single quote in this string!"  
>>> print("I'm using a single quote in this string!")  
I'm using a single quote in this string!
```

String Literals (2 of 2)

- Use `' '` and `"""` for multi-line paragraphs

```
>>> print("""This very long sentence extends  
all the way to the next line. """)  
This very long sentence extends  
all the way to the next line.
```

- When you evaluate a string in Python without the `print` function, you can see the literal for the **newline character**, `\n`, embedded in the result:

```
>>> """This very long sentence extends  
all the way to the next line."""  
'This very long sentence extends\nall the way to the next line.'
```

Escape Sequences

- The newline character `\n` is called an **escape sequence**

Escape Sequence	Meaning
<code>\b</code>	Backspace
<code>\n</code>	Newline
<code>\t</code>	Horizontal tab
<code>\\</code>	The <code>\</code> character
<code>\'</code>	Single quotation mark
<code>\"</code>	Double quotation mark

String Concatenation

- You can join two or more strings to form a new string using the concatenation operator +

```
>>> "Hi " + "there, " + "Ken!"  
'Hi there, Ken!'
```

- The * operator allows you to build a string by repeating another string a given number of times

```
>>> " " * 10 + "Python"  
'          Python'
```

Variables and the Assignment Statement

(1 of 3)

- A **variable** associates a name with a value
 - Makes it easy to remember and use later in the program
- Variable naming rules:
 - Reserved words cannot be used as variable names
 - Examples: `if`, `def`, and `import`
 - Name must begin with a letter or `_`
 - Name can contain any number of letters, digits, or `_`
 - Names are case sensitive
 - Tip: Begin each word in the variable name with an uppercase, except the first one (Example: `interestRate`)

Variables and the Assignment Statement (2 of 3)

- Programmers use all uppercase letters for **symbolic constants** (variables that contain values that the program never changes)
 - Examples: TAX_RATE and STANDARD_DEDUCTION
- Variables receive initial values and can be reset to new values with an **assignment statement**
<variable name> = <expression>
 - Subsequent uses of the variable name in expressions are known as **variable references**
- Example:

```
>>> firstName = "Ken"
>>> secondName = "Lambert"
>>> fullName = firstName + " " + secondName
>>> fullName
'Ken Lambert'
```

Variables and the Assignment Statement

(3 of 3)

- Variables serve two purposes:
 - Help the programmer keep track of data that change over time
 - Allow the programmer to refer to a complex piece of information with a simple name (a process called **abstraction**)

Activity 2.1: Knowledge Check

1. What is a literal?
2. Which operator allows you to build a string by repeating another string a given number of times?
3. Which types of characters are allowed in variable names?

Program Comments and Docstrings (1 of 3)

- **Program comment**

- A piece of program text that the computer ignores but that provides useful documentation

- **Docstring**

- A multi-line string of the form:

```
"""
```

```
Program: circle.py
```

```
Author: Ken Lambert
```

```
Last date modified: 10/10/22
```

```
The purpose of this program is to compute the area of a circle. The input  
is an integer or floating-point number representing the radius of the  
circle. The output is a floating-point number labeled as the area of the  
circle.
```

```
"""
```

Program Comments and Docstrings (2 of 3)

- **End-of-line comments**

- Begin with the # symbol and extend to the end of a line
- Might explain the purpose of a variable or the strategy used by a piece of code
- Example:

```
>>> RATE = 0.70 # Conversion rate for Canadian to US  
dollars
```

Program Comments and Docstrings (3 of 3)

- Tips related to comments and docstrings:
 - Begin a program with a statement of purpose and other information helpful to programmers
 - Accompany a variable definition with a comment that explains the variable's purpose
 - Precede major segments of code with brief comments that explain their purpose
 - Include comments to explain the workings of complex or tricky sections of code

Numeric Data Types and Character Sets

- The first applications of computers were to crunch numbers
- The use of numbers in many applications is still very important

Integers

- In real life, the range of **integers** (whole numbers) is infinite
- Integer literals are written without commas
- A computer's memory places a limit on the magnitude of the largest positive and negative integers
- The most common `int` implementation's typical range:
 -2^{31} to $2^{31} - 1$
- Python's integer is much larger and is limited only by the memory of the computer

Floating-Point Numbers (1 of 2)

- Real numbers have **infinite precision**
 - The digits in the fractional part can continue forever
- Python uses **floating-point** numbers to represent real numbers
- Python's `float` typical range:
 -10^{308} to $10^{308} - 1$
- Typical precision: 16 digits

Floating-Point Numbers (2 of 2)

- A floating-point number can be written using either ordinary **decimal notation** or **scientific notation**

Decimal Notation	Scientific Notation	Meaning
3.78	3.78e0	3.78×10^0
37.8	3.78e1	3.78×10^1
3780.0	3.78e3	3.78×10^3
0.378	3.78e-1	3.78×10^{-1}
0.00378	3.78e-3	3.78×10^{-3}

Expressions

- A literal evaluates to itself
- A variable reference evaluates to the variable's current value
- **Expressions** provide an easy way to perform operations on data values to produce other values
- When an expression is entered at the Python shell prompt:
 - The expression's operands are evaluated
 - Its operator is then applied to these values to compute the value of the expression

Arithmetic Expressions (1 of 3)

- An **arithmetic expression** consists of combined operands and operators

Operator	Meaning	Syntax
–	Negation	$-a$
**	Exponentiation	$a ** b$
*	Multiplication	$a * b$
/	Division	a / b
//	Quotient	$a // b$
%	Remainder or modulus	$a \% b$
+	Addition	$a + b$
–	Subtraction	$a - b$

Arithmetic Expressions (2 of 3)

- **Precedence rules:**

- `**` has the highest precedence and is evaluated first
- Unary negation is evaluated next
- `*`, `/`, and `%` are evaluated before `+` and `-`
- `+` and `-` are evaluated before `=`
- With two exceptions, operations of equal precedence are left associative, so they are evaluated from left to right
 - `**` and `=` are right associative
- You can use `()` to change the order of evaluation

Arithmetic Expressions (3 of 3)

- When both operands of an expression are of the same numeric type, the resulting value is also of that type
- When each operand is of a different type, the resulting value is of the more general type
- `//` produces an integer quotient; `/` produces a float
 - Example: `3 // 4` is 0, whereas `3 / 4` is .75
- For multi-line expressions, use a `\`
- Example:

```
>>> 3 + 4 * \
2 ** 5
131
```

Mixed-Mode Arithmetic and Type Conversions (1 of 4)

- **Mixed-mode arithmetic** involves integers and floating-point numbers:

```
>>> 3.14 * 3 ** 2
```

```
28.26
```

- You must use a **type conversion function** when working with the input of numbers
 - It is a function with the same name as the data type to which it converts
- `input` function returns a string as its value
 - You must use the `int` or `float` function to convert the string to a number before performing arithmetic

Mixed-Mode Arithmetic and Type Conversions (2 of 4)

Conversion Function	Example Use	Value Returned
<code>int(<a number or a string>)</code>	<code>int(3.77)</code> <code>int("33")</code>	3 33
<code>float(<a number or a string>)</code>	<code>float(22)</code>	22.0
<code>str(<any value>)</code>	<code>str(99)</code>	'99'

Mixed-Mode Arithmetic and Type Conversions (3 of 4)

- Note that the `int` function converts a `float` to an `int` by truncation, not by rounding

```
>>> int(6.75)
```

```
6
```

```
>>> round(6.75)
```

```
7
```

Mixed-Mode Arithmetic and Type Conversions (4 of 4)

- Type conversion also occurs in the construction of strings from numbers and other strings

```
>>> profit = 1000.55
```

```
>>> print('$' + profit)
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
TypeError: cannot concatenate 'str' and 'float' objects
```

- **Solution: use str function**

```
>>> print('$' + str(profit))
```

```
$1000.55
```

- Python is a **strongly typed programming language**

Using Functions and Modules

- Python includes many useful functions, which are organized in libraries of code called **modules**

Calling Functions: Arguments and Return Values

- A **function** is chunk of code that can be called by name to perform a task
- Functions often require **arguments**, referred to by names known as **parameters**
 - Arguments may be **optional** or **required**
- When a function completes its task, it may **return a value** back to another part of the program
- To learn how to use a function's arguments, use the `help` function:

```
>>> help(round)
Help on built-in function round in module builtin:
round(...)
    round(number[, ndigits]) -> floating point number
    Round a number to a given precision in decimal digits (default 0 digits).
    This returns an int when called with one argument, otherwise the same type as
    number, ndigits may be negative.
```

The math Module (1 of 2)

- Functions and other resources are coded in components called modules
- Functions like `abs` and `round` from the `_builtin_` module are always available to use
- The `math` module includes functions that perform basic mathematical operations
- To use a resource from a module, you write the name of a module as a qualifier, followed by a dot (.) and the name of the resource
 - Example: `math.pi`

```
>>> math.pi  
3.1415926535897931  
>>> math.sqrt(2)  
1.4142135623730951
```

The math Module (2 of 2)

- You can avoid the use of the qualifier with each reference by importing the individual resources

```
>>> from math import pi, sqrt
>>> print(pi, sqrt(2))
3.14159265359 1.41421356237
```

- You may import all of a module's resources to use without the qualifier
 - Example: `from math import *`

The Main Module

- You can write documentation for a Python script
- To differentiate this script from the other modules in a program, we call it the **main module**
 - Like any module, the main module can be imported

```
>>> import taxform
Enter the gross income: 120000
Enter the number of dependents: 2
The income tax is $20800.0
```
- After importing a main module, view its documentation by running the `help` function

Program Format and Structure

- Start with a comment including the author's name, the purpose of the program, and other relevant information
 - In a docstring
- Then, include statements that:
 - Import any modules needed by program
 - Initialize important variables, suitably commented
 - Prompt the user for input data and save the input data in variables
 - Process the inputs to produce the results
 - Display the results

Running a Script from a Terminal Command Prompt (1 of 4)

- A way to run a Python script:
 - Open a terminal command prompt window
 - Click in the “Type here to search” box, type Command Prompt, and click Command Prompt in the list
 - Navigate or change directories until the prompt shows the directory that contains the Python script
 - Run the script by entering:
 - `python3 scriptname.py`

Running a Script from a Terminal Command Prompt (2 of 4)



Figure 2-5 A terminal window on a Macintosh

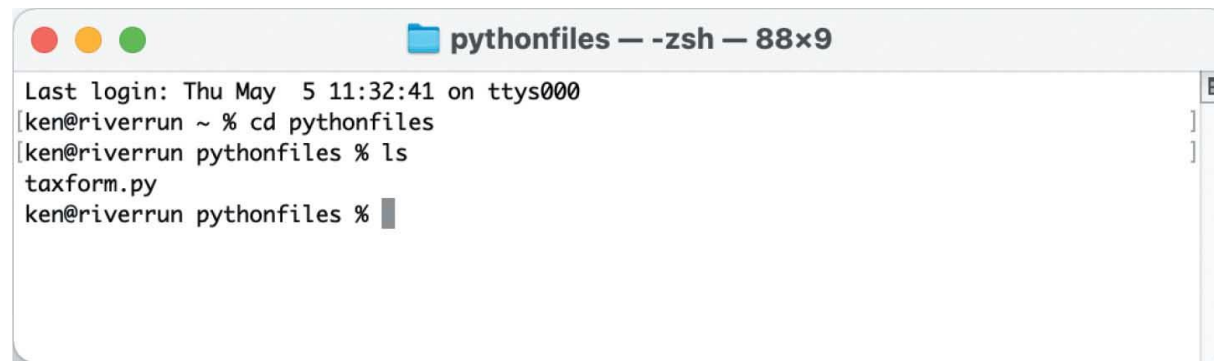


Figure 2-6 Changing to another directory and listing its contents

Running a Script from a Terminal Command Prompt (3 of 4)

A screenshot of a terminal window titled "pythonfiles — -zsh — 88x9". The window shows a sequence of commands and their outputs. The user logs in, changes to the "pythonfiles" directory, lists the files (showing "taxform.py"), and then runs "python3 taxform.py". The script prompts for "gross income" (42000) and "number of dependents" (2), and outputs "The income tax is \$5200.0".

```
Last login: Thu May  5 11:32:41 on ttys000
[ken@riverrun ~ % cd pythonfiles
[ken@riverrun pythonfiles % ls
taxform.py
[ken@riverrun pythonfiles % python3 taxform.py
Enter the gross income: 42000
Enter the number of dependents: 2
The income tax is $5200.0
ken@riverrun pythonfiles %
```

Figure 2-7 Running a Python script in a terminal window

Running a Script from a Terminal Command Prompt (4 of 4)

- Python installations enable you to launch Python scripts by double-clicking the files from the OS's file browser
 - May require `.py` file type to be set
 - Fly-by-window problem: window will close automatically
 - Solution: Add an input statement at end of the script that pauses until the user presses the enter or return key

```
>>> input("Please press enter or return to quit the program. ")
```